

Train models

The MicroICS “Train models” option has two modes:

- Evaluation of performance and training of several machine learning models (the terms model, classifier, and learner are used interchangeably) using the “all learners” setting.
- Training a single model of your choice.

When running the pipeline for the first time, it is worth training all learners – it takes longer, but it shows you which model performs best on your dataset. The trained models are saved so that you can later use them as an inference model to classify unknown images.

Random forest is often the best-performing model, and for this we provide additional outputs on feature ranking and ablation, described below.

The quality of the result depends on supplying a high-quality dataset (imaging data) with sufficient replication. Model training and evaluation are performed together using stratified cross-validation as the main evaluation procedure. All images in the dataset are split into n sub-datasets (folds). In each training run, $n-1$ folds are used to train the model and the held-out fold is used to evaluate performance by running predictions on the held-out data. The held-out fold is always different. For example, if the dataset is split into 5 folds, training is run 5 times, each time using a different combination of folds for training and a different fold for evaluation. The average score across repetitions is then reported. This process is repeated for each classifier, allowing direct comparison across classifiers and showing which is most suitable for your dataset and biological question.

For training to run, each class must contain enough images to be split into training and prediction subsets, taking into account the unit of replication and the number of images per class. If any class has too few images, the software raises an error. Additionally, the classes should be approximately equal in size: large imbalances bias the model towards the better-represented classes and reduce accuracy for the under-represented ones.

Understanding the benchmark output

This section explains what each training output file contains and how to read it, followed by a real dataset case study on interpreting the features that the model identified as key for the given biological question.

With respect to the image file naming convention, on which the pipeline relies to read the label from the image name, the label and the date are of great importance for training, as described below.

The label is encoded in the image name and is directly associated with the biological property the classifier is trained to predict. By default, this is strain identity, taken from the string following `\_st\_` in the filename (see resources/Image requirements). When you change the word (string) that follows `\_st\_`, you change the label and consequently what the model learns to distinguish, but the way the pipeline is run does not change.

To use the same dataset for a different biological question (when possible), you need to link the image to a new label by replacing the strain name after `\_st\_` with a label representing your property of interest, then run the pipeline as usual.

For example, to associate biofilm structure with clinical relevance, replace the strain name with CI (clinically relevant) or CNI (clinically not relevant), according to the epidemiology of each strain:

Strain L628 is clinically relevant, so image name

13042023_s_Lm_st_L628_p_C06_pos001_tm_24_ch_Syto9_z_21

becomes 13042023_s_Lm_st_CI_p_C06_pos001_tm_24_ch_Syto9_z_21

Strain L1764 is not clinically relevant, so

13062023_s_Lm_st_L1764_p_G05_pos002_tm_24_ch_Syto9_z_21

becomes 13062023_s_Lm_st_CNI_p_G05_pos002_tm_24_ch_Syto9_z_21.

With the images re-labelled in this way, the model reads the label CN or CNI to learn to distinguish clinically relevant from non-relevant biofilms, using the same structural features and the same pipeline.

From the file name, the date (the first eight characters from the start) is important in terms of batch-induced variability. During training, alongside the label, the date is also used as a target to evaluate which features are important for predicting the date and can be understood as information on which features are important for batch effect.

Output file explanations

- **classification_all and classification_no_counts_features:** These summarise the classification results for each algorithm. The columns, from left to right:

The columns in the table:

tag – always RESULT; used for parsing, currently inactive.

model – the classifier type (random forest, and so on), plus a dummy classifier that serves as the baseline (see below).

upsampling – currently always 1.

n_components – the number of input features. Classifiers can be given all features (currently about 2,700), or a reduced set produced in advance by SVD. SVD creates components as linear combinations of the original features and ranks them, and during evaluation incrementally reduces the number of components from the entire data set down to 16 to determine whether a reduced number of input features is beneficial for the performance of the tested classifiers.

accuracy – the proportion of correct predictions; the key result of the comparison.

test_set – the labels (for example strain names) used in the classification test.

thr_features – TRUE if threshold-based features are included, FALSE if excluded. For each n_components, both are given; for n_components 580 only FALSE, and for 2700 only TRUE.

The remaining files are derived from the random forest classification only:

- **ablation_ranking:** This plot shows how model accuracy changes as features are incrementally added.
- **rankings_label and rankings_date:** For each feature from all features calculated, these files give the strength of its relationship to the target: larger values indicate that the feature was

more important in predicting the strain (`rankings_label`), and likewise for the date (`rankings_date`). To identify which features were important for the biological question of interest, use `rankings_label` and select the 100 most important features for further analysis. Additional files are provided in the visualisations to support interpretation of the features.

We provide the date ranking to account for features that vary because the experiment was repeated on different days – a batch effect, usually arising from the methodology or from natural variability in the biological system. This matters whenever you supply images from several days (biological replicates) for one strain, because variation within a single day is often smaller than variation between days. If the model leans on features tied to specific dates rather than to strains across dates, the result is biased and less reliable. We therefore recommend comparing `rankings_label` and `rankings_date`, and setting aside any high-ranking features that appear in both when interpreting the biology.

Visualisations:

- The performance comparison across classifiers (from **`classification_all`** and **`classification_no_counts`**) and the ablation ranking (**`ablation_ranking`**) are also provided as plots in the Visualisations folder to make interpretation easier.
- **Confusion matrix.** A matrix of true vs predicted class for the given dataset. Rows represent the true class, columns the predicted class; the diagonal shows correct predictions, and off-diagonal cells show errors. From this matrix you can identify which classes are easy to separate and which are confused with each other, implying that with the given set of features and model they are hard to distinguish.
- **Top_feature_correlations.** Correlation among the top features used to assess redundancy. Features are clustered according to their correlations, based on their values and the given cutoff. Features that fall within the same cluster may carry the same feature signal and thus be redundant. Examine the clusters and identify which features occur together. If the feature family imply the same trends, the clusters can be used for interpretation rather than individual features.
- **Feature values.** The values of the top features are visualised for each class to help clarify the differences between the classes. The grey dots show individual measurements, the orange diamonds indicate the averages, and the box plots represent the median and interquartile range.

Focusing on the features that explain what distinguishes your classes

General approach

To elaborate on the conclusions about the features driving the model decisions for biological hypotheses, we provide the following framework. Look for named MicroICS outputs and perform additional analysis on the provided calculations when required; you may also use supporting scripts provided at <https://github.com/nikajanez/Microics-results-analysis>.

- From `rankings_label`, select the top 100 features.
- Compare these with `rankings_date` and mark any that overlap, as these capture the imaging day as well as the biology.
- Check how the features are calculated, per image or per layer; in the 3D image the vertical profile is usually very important. Use the description of the features provided in the resources.
- Check the correlation heatmap and determine how many clusters the top features can be divided into, based on the given cutoff. Interpret the clusters, also taking into account the type of features and the aggregation types, and compare them in the context of aggregation; for example, means and standard deviations carry very different information.
- Identify what the features in the clusters are measuring, using the provided description of the features resources/`MicroICS_calculated_features`.

Then select representative images from all classes in the dataset and inspect them in terms of the observed trends in the features. Combine the information from the top features with the corresponding observations in the images, and formulate a hypothesis about what may be driving these patterns.

Case study

Here we provide a description of the case study from Janež, Škrlić, Osojnik et al. 2026 (doi: 10.1038/s41522-026-01083-8) for the task of distinguishing between strains (identity) and explain how to combine information on top features with observations from images, which may be less intuitive but becomes clearer with experience.

Main observations from the feature ranking:

- Layers are extremely important; all top features are calculated per layer.
- Geometric complexity (fractal dimension, correlation) of the structure and space with below-average intensity (`minprop`), relating to empty space or weak signals, and how this changes through the 3D layers, is important. We need to assess whether the biofilm appears ordered, with a repeating pattern, or disordered, with no pattern, and how this differs between layers.
- The number of bacteria on each layer and how these changes through the 3D layers (`counts`, `GPT volume`).
- Brightness of bacteria (intensity-based features). We need to assess whether bacteria are bright or dim, and how this varies within and between layers. Are there a few bright ones and many dim ones, or vice versa? Are bacteria similarly bright in one layer or across all layers, or are they brighter at the bottom and dimmer in the upper layers?
- The aggregations are mostly at extreme or near-extreme values rather than around central tendencies, so the peaks (both low and high) are important for distinguishing the strains. Thus, we focus on observations that stand out, not solely on the average impression.

Comparing the strains based on representative images (five per strain):

ATCC 19115

- Visually one extreme.
- At the bottom it is dense, confluent, with a low void volume in terms of spaces between bacteria; it seems to have a high count.
- The bacteria stop appearing after the 11th layer in most repetitions.
- Bacteria tend to form “pyramids” originating from some bacteria at the bottom.
- In the confusion matrix: 98.4% accuracy.

L628

- Visually sits closest to ATCC 19115.
- At the bottom it is dense, nearly confluent, with resolvable cells, uniformly bright.
- In the upper layers there is no lawn but aggregates with larger voids between them, and the number and size of aggregates decreases from the bottom to the top. The aggregates are larger than in L1764.
- Confusion matrix: 84% accuracy, mostly mixed with L455.

L455

- Sits next to L628 at the ATCC 19115 end.
- At the bottom it is dense, with well-resolved cells, nearly confluent, low void volume.
- In the upper layers it is similar to L628; it is hard to say whether there is much difference from L628, L634, L1323 or L394.
- Confusion matrix: 82.4% accuracy; it is mixed with L628, L634 and L1323.

L634

- Sits next to L455 at the ATCC 19115 end.
- At the bottom it is also dense, bright, with low void volume.
- In the upper layers it is similar to L628 and the other strains at this end, but not to L1764.
- Confusion matrix: 81.6% accuracy, mostly mixed with L1323, which comes next in rank at the ATCC 19115 end.

L1323

- Sits next to L634 at the ATCC 19115 end.
- At the bottom it is dense, and part of the population is slightly dimmer than ATCC 19115; it is closer to L1764 but still has the carpet-like pattern of ATCC 19115, with a slightly larger void volume.
- In the upper layers it remains similar to L628.
- Confusion matrix: 79.2% accuracy, mixed with L634 and L455.

L1764

- Visually the other extreme in comparison with ATCC 19115.
- At the bottom there are very small clusters; many bacteria are dim and create the illusion of mist around the bacteria, but they are actually clumps of bacteria of different intensities. They are spread throughout the entire volume of the image; the bottom has the highest proportion of bright bacteria.
- In the confusion matrix: 92% accuracy.

Conclusions based on observations: the identity of a strain from 3D structure depends on how densely packed the biomass is and how uniform it is within the 3D volume, rather than on the total amount of the biofilm. The eight classes are distinguished by how they occupy the space of the 3D image, the size of aggregates, how these aggregates are distributed throughout the volume, the brightness of bacteria, how the biomass disappears across layers, and how much empty space lies between the aggregates.